

SIMATS ENGINEERING
ASSESSMENT TOOL 4 - HIGH
(ANALYTICAL PROBLEM SOLVING QUESTIONS)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO3: Data Frames, Making data frames - Working with data frames- Data Reshaping- Melting and Casting of data – Merging Data Frames - Editing and Reading Data from Files – Reading and Writing Files.
(BL3, BL5)

Assessment Tool Weightage: 15%

QUESTIONS: 10

Total Marks: 100

ANNEXURE A

ANALYTICAL PROBLEM SOLVING QUESTIONS

Code of Conduct:

I TANUJA P (192321052) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: tanuja p

1. Student Performance Analysis (10 Marks)

A CSV file contains student details with columns: StudentID, Name, Department, Subject, Marks1, Marks2, Marks3.

Write an R program to:

- Read the CSV file.
- Create a new column Total and Average.
- Identify students scoring above the class average.
- Display department-wise average marks.

```
students <- read.csv("D:/Student.csv")
print("Student Details")
print(students)
students$Total <- students$Marks1 +
  students$Marks2 +
  students$Marks3
students$Average <- students$Total / 3
print("Student Details with Total and Average")
print(students)
class_avg <- mean(students$Average)
print("Class Average")
print(class_avg)
above_avg <- students[students$Average > class_avg, ]
print("Students Scoring Above Class Average")
print(above_avg)
dept_avg <- aggregate(Average ~ Department,
  data = students,
  mean)
print("Department-wise Average Marks")
print(dept_avg)
```

```
> print(class_avg)
[1] "Class Average"
> print(class_avg)
[1] 81.93333
> above_avg <- students[students$Average > class_avg, ]
> print("Students Scoring Above Class Average")
[1] "Students Scoring Above Class Average"
> print(above_avg)
```

	StudentID	Name	Department	Subject	Marks1	Marks2	Marks3	Total	Average
2	102	Priya	IT	Maths	95	90	85	270	90
4	104	Meena	CSE	Maths	88	92	84	264	88

```
> dept_avg <- aggregate(Average ~ Department,
+                         data = students,
+                         mean)
> print("Department-wise Average Marks")
[1] "Department-wise Average Marks"
> print(dept_avg)
```

	Department	Average
1	CSE	84.83333
2	ECE	70.00000
3	IT	85.00000

2. Employee Salary Management (10 Marks)

An organization maintains two datasets:

- Employee.csv (EmpID, Name, Department)
- Salary.csv (EmpID, BasicPay, Allowance)

Write an R program to:

- Merge both datasets.
- Calculate Gross Salary.
- Find the department with the highest average salary.
- Display employees earning above ₹60,000.

```
employee <- read.csv("D:/Employee.csv")
salary <- read.csv("D:/Salary.csv")
print("Employee Details")
print(employee)
print("Salary Details")
print(salary)
emp_data <- merge(employee,
                  salary,
                  by="EmpID")
print("Merged Dataset")
print(emp_data)
emp_data$GrossSalary <-
  emp_data$BasicPay +
  emp_data$Allowance
print("Employee Gross Salary")
print(emp_data)
dept_avg <- aggregate(GrossSalary ~ Department,
                     data=emp_data,
                     mean)
print("Department-wise Average Salary")
print(dept_avg)
highest <- dept_avg[
  which.max(dept_avg$GrossSalary), ]
print("Highest Average Salary Department")
print(highest)
high_salary <-
  emp_data[emp_data$GrossSalary > 60000, ]
print("Employees earning above Rs.60000")
print(high_salary)
```

```

.                                     data emp_data,
+                                     mean)
> print("Department-wise Average Salary")
[1] "Department-wise Average Salary"
> print(dept_avg)
  Department GrossSalary
1   Finance      55000
2        HR      49500
3        IT      74000
> highest <- dept_avg[
+   which.max(dept_avg$GrossSalary), ]
> print("Highest Average Salary Department")
[1] "Highest Average Salary Department"
> print(highest)
  Department GrossSalary
3        IT      74000
> high_salary <-
+   emp_data[emp_data$GrossSalary > 60000, ]
> print("Employees earning above Rs.60000")
[1] "Employees earning above Rs.60000"
> print(high_salary)
  EmpID  Name Department BasicPay Allowance GrossSalary
2  E102 Priya         IT   55000   15000      70000
4  E104 Meena         IT   60000   18000      78000
>
>

```

3. Sales Data Reshaping (10 Marks)

A sales dataset stores monthly sales of products in separate columns (Jan, Feb, Mar, Apr).

Write an R program to:

- Convert the data from wide format to long format using melt().
- Calculate total sales for each product.
- Convert the data back into wide format using dcast().

```

library(reshape2)
sales <- read.csv("D:/Sales.csv")
print("Original Data")
print(sales)
long_data <- melt(sales,
                  id.vars="Product")

print("Long Format")
print(long_data)
total_sales <- aggregate(value ~ Product,
                        data=long_data,
                        sum)

print("Total Sales")
print(total_sales)

```

```

wide_data <- dcast(long_data,
                  Product ~ variable,
                  value.var="value")

print("Wide Format")
print(wide_data)
> print("Original Data")
[1] "Original Data"
> print(sales)
  Product   Jan   Feb   Mar   Apr
1  Laptop 10000 12000 11000 13000
2  Mobile  8000  9000 10000  9500
3  Tablet  5000  6000  5500  6500
> long_data <- melt(sales,
+                   id.vars="Product")
>
> print("Long Format")
[1] "Long Format"
> print(long_data)
  Product variable value
1  Laptop      Jan 10000
2  Mobile      Jan  8000
3  Tablet      Jan  5000
4  Laptop      Feb 12000
5  Mobile      Feb  9000
6  Tablet      Feb  6000
7  Laptop      Mar 11000
8  Mobile      Mar 10000
9  Tablet      Mar  5500
10 Laptop      Apr 13000
11 Mobile      Apr  9500
12 Tablet      Apr  6500

```

4. Hospital Patient Records (10 Marks)

Hospital records contain patient information stored in two different files:

- Patient Details
- Treatment Details

Write an R program to:

- Read both files.
- Merge them using Patient ID.
- Find patients who visited more than twice.
- Save the final report into a CSV file.

```
patient <- read.csv("D:/Patient.csv")
```

```
treatment <- read.csv("D:/Treatment.csv")

print("Patient Details")
print(patient)

print("Treatment Details")
print(treatment)

hospital <- merge(patient,
                  treatment,
                  by="PatientID")

print("Merged Dataset")
print(hospital)

visit_count <- aggregate(PatientID ~ PatientID,
                        data=hospital,
                        length)

print("Visit Count")
print(visit_count)

more_visit <- visit_count[
  visit_count$PatientID > 2, ]

print("Patients Visited More Than Twice")
print(more_visit)

write.csv(hospital,
          "HospitalReport.csv",
          row.names=FALSE)

print("Hospital Report Saved")
```

```

;      P101      Arun  25  Male  10-01-2026  Injection
;      P102      Priya 30 Female 02-01-2026    Cold
;      P103      Ravi  40  Male  03-01-2026  Diabetes
;      P104      Meena 35 Female 04-01-2026    Fever
;      P104      Meena 35 Female 06-01-2026  Checkup
;      P104      Meena 35 Female 08-01-2026  Medicine
;
; visit_count <- aggregate(PatientID ~ PatientID,
;                           data=hospital,
;                           length)
;
; print("Visit Count")
;1] "visit Count"
; print(visit_count)
; PatientID
;      8
;
; more_visit <- visit_count[
;   visit_count$PatientID > 2, ]
;
; print("Patients Visited More Than Twice")
;1] "Patients Visited More Than Twice"
; print(more_visit)
;1] 8
;
; write.csv(hospital,
;           "HospitalReport.csv",
;           row.names=FALSE)
;
; print("Hospital Report Saved")
;1] "Hospital Report Saved"
;

```

5. Cricket Statistics Analysis (10 Marks)

A dataset contains Player Name, Match, Runs, Strike Rate, Venue.

Write an R program to:

- Read the dataset.
- Create a new data frame containing only players with Strike Rate above 140.
- Calculate average runs venue-wise.
- Export the filtered data into a CSV file.

```
cricket <- read.csv("D:/Cricket.csv")
```

```
print("Cricket Dataset")
```

```
print(cricket)
```

```
filtered <- cricket[
  cricket$StrikeRate > 140, ]
```

```
print("Players with Strike Rate Above 140")
```

```
print(filtered)
```

```
venue_avg <- aggregate(Runs ~ Venue,
                       data=cricket,
                       mean)
```

```
print("Venue-wise Average Runs")
print(venue_avg)
```

```
write.csv(filtered,
           "HighStrikeRate.csv",
           row.names=FALSE)
```

```
print("Filtered Data Exported Successfully")
```

```
> print(filtered)
  PlayerName Match Runs StrikeRate Venue
1    Virat     1   85      145 Chennai
2    Rohit     2  102      152  Mumbai
4    Rahul     4   70      142  Mumbai
5   Hardik     5   48      160 Chennai
>
> venue_avg <- aggregate(Runs ~ Venue,
+                         data=cricket,
+                         mean)
>
> print("Venue-wise Average Runs")
[1] "Venue-wise Average Runs"
> print(venue_avg)
  Venue Runs
1 Chennai 66.5
2  Delhi 55.0
3  Mumbai 86.0
>
> write.csv(filtered,
+           "HighStrikeRate.csv",
+           row.names=FALSE)
>
> print("Filtered Data Exported Successfully")
[1] "Filtered Data Exported Successfully"
>
```

6. Online Shopping Order Analysis (10 Marks)

Two datasets contain customer details and order details.

Write an R program to:

- Merge the datasets.
- Identify customers with more than three purchases.
- Calculate total purchase amount customer-wise.
- Save the report as an Excel or CSV file.

```
customer <- read.csv("D:/Customer.csv")
```

```
orders <- read.csv("D:/Orders.csv")
```

```
print("Customer Details")
```



```
print(customer)

print("Order Details")
print(orders)

shopping <- merge(customer,
                  orders,
                  by="CustomerID")

print("Merged Dataset")
print(shopping)

purchase_count <- aggregate(OrderID ~ CustomerID,
                             data=shopping,
                             length)

print("Purchase Count")
print(purchase_count)

more_purchase <- purchase_count[
  purchase_count$OrderID > 3, ]

print("Customers with More Than 3 Purchases")
print(more_purchase)

total_purchase <- aggregate(Amount ~ CustomerID,
                             data=shopping,
                             sum)

print("Total Purchase Amount")
print(total_purchase)

write.csv(total_purchase,
          "ShoppingReport.csv",
          row.names=FALSE)

print("Shopping Report Saved")
```

```

>
> purchase_count <- aggregate(OrderID ~ CustomerID,
+                             data=shopping,
+                             length)
>
> print("Purchase Count")
[1] "Purchase Count"
> print(purchase_count)
  CustomerID OrderID
1         C101         4
2         C102         1
3         C103         1
4         C104         1
>
> more_purchase <- purchase_count[
+   purchase_count$OrderID > 3, ]
>
> print("Customers with More Than 3 Purchases")
[1] "Customers with More Than 3 Purchases"
> print(more_purchase)
  CustomerID OrderID
1         C101         4

```

7. Weather Data Transformation (10 Marks)

A weather dataset contains daily temperature values stored as separate columns for each city.

Write an R program to:

- Reshape the data using melt().
- Calculate average temperature city-wise.
- Convert the reshaped data back using dcast().
- Write the output into a new CSV file.

```
library(reshape2)
```

```
weather <- read.csv("D:/Weather.csv")
```

```
print("Original Weather Data")
```

```
print(weather)
```

```
long_data <- melt(weather,
  id.vars="Day")
```

```
print("Long Format")
```

```
print(long_data)
```

```
avg_temp <- aggregate(value ~ variable,
  data=long_data,
  mean)
```

```
print("Average Temperature")
```

```
print(avg_temp)
```

```
wide_data <- dcast(long_data,  
  Day ~ variable,  
  value.var="value")
```

```
print("Wide Format")  
print(wide_data)
```

```
write.csv(wide_data,  
  "WeatherOutput.csv",  
  row.names=FALSE)
```

```
print("Weather Report Saved")
```

```
> print("Average Temperature")
```

```
[1] "Average Temperature"
```

```
> print(avg_temp)
```

```
variable value
```

```
1 Chennai 34.75
```

```
2 Mumbai 31.75
```

```
3 Delhi 30.50
```

```
>
```

```
> wide_data <- dcast(long_data,  
+   Day ~ variable,  
+   value.var="value")  
>
```

```
> print("Wide Format")
```

```
[1] "Wide Format"
```

```
> print(wide_data)
```

```
Day Chennai Mumbai Delhi
```

```
1 Mon      34      31     29
```

```
2 Thu      34      31     32
```

```
3 Tue      35      32     30
```

```
4 Wed      36      33     31
```

```
>
```

```
> write.csv(wide_data,  
+   "WeatherOutput.csv",  
+   row.names=FALSE)  
>
```

```
> print("Weather Report Saved")
```

```
[1] "Weather Report Saved"
```

8. Banking Transaction Analysis (10 Marks)

Bank maintains two files:

- Customer Details
- Transaction Details

Write an R program to:

- Merge the files.
- Calculate total transaction amount for each customer.
- Identify customers with transactions exceeding ₹1,00,000.
- Export the result.

```
customer <- read.csv("D:/Customer.csv")
```

```
transaction <- read.csv("D:/Transaction.csv")
```

```
print("Customer Details")  
print(customer)
```

```
print("Transaction Details")  
print(transaction)
```

```
bank <- merge(customer,  
              transaction,  
              by="CustomerID")
```

```
print("Merged Dataset")  
print(bank)
```

```
total_amount <- aggregate(Amount ~ CustomerID,  
                           data=bank,  
                           sum)
```

```
print("Total Transaction Amount")  
print(total_amount)
```

```
high_transaction <- total_amount[  
  total_amount$Amount > 100000, ]
```

```
print("Customers Exceeding Rs.100000")  
print(high_transaction)
```

```
write.csv(high_transaction,  
          "BankReport.csv",  
          row.names=FALSE)
```

```
print("Bank Report Saved")
```

```

> print("Transaction Details")
[1] "Transaction Details"
> print(transaction)
  CustomerID TransactionID Amount
1      C101           T1  50000
2      C101           T2  60000
3      C102           T3  30000
4      C103           T4 120000
>
> bank <- merge(customer,
+               transaction,
+               by="CustomerID")
>
> print("Merged Dataset")
[1] "Merged Dataset"
> print(bank)
  CustomerID CustomerName      City TransactionID Amount
1      C101      Arun      Chennai           T1  50000
2      C101      Arun      Chennai           T2  60000
3      C102     Priya Bangalore           T3  30000
4      C103      Ravi Hyderabad           T4 120000
~

```

9. University Attendance Report (10 Marks)

Attendance data is available in multiple CSV files.

Write an R program to:

- Read all files.
- Combine them into a single data frame.
- Calculate attendance percentage.
- Identify students with attendance below 75%.
- Save the result.

```
attendance1 <- read.csv("D:/Attendance1.csv")
```

```
attendance2 <- read.csv("D:/Attendance2.csv")
```

```
attendance <- rbind(attendance1,
                    attendance2)
```

```
print("Combined Attendance")
```

```
print(attendance)
```

```
attendance$Percentage <-
  (attendance$PresentDays /
   attendance$WorkingDays) * 100
```

```
print("Attendance Percentage")
```

```
print(attendance)
```

```
low_attendance <-
```

```

attendance[attendance$Percentage < 75, ]

print("Students Below 75 Percent")
print(low_attendance)

write.csv(low_attendance,
          "AttendanceReport.csv",
          row.names=FALSE)

print("Attendance Report Saved")

> print("Attendance Percentage")
[1] "Attendance Percentage"
> print(attendance)
  StudentID  Name PresentDays WorkingDays Percentage
1         101  Arun          70           90  77.77778
2         102 Priya          80           90  88.88889
3         103  Ravi          60           90  66.66667
4         104 Meena          88           90  97.77778
>
> low_attendance <-
+   attendance[attendance$Percentage < 75, ]
>
> print("Students Below 75 Percent")
[1] "Students Below 75 Percent"
> print(low_attendance)
  StudentID Name PresentDays WorkingDays Percentage
3         103  Ravi          60           90  66.66667
>
> write.csv(low_attendance,
+           "AttendanceReport.csv",
+           row.names=FALSE)
>
> print("Attendance Report Saved")
[1] "Attendance Report Saved"

```

10. Movie Rating Analysis (10 Marks)

A dataset contains Movie Name, Genre, Viewer Rating, Platform.

Write an R program to:

- Read the dataset.
- Calculate average rating for each genre.
- Reshape the dataset using melt and cast operations.
- Export the transformed dataset.

```
library(reshape2)
```

```
movie <- read.csv("D:/Movie.csv")
```

```
print("Movie Dataset")
print(movie)

genre_avg <- aggregate(ViewerRating ~ Genre,
                        data=movie,
                        mean)

print("Genre-wise Average Rating")
print(genre_avg)

long_data <- melt(movie,
                  id.vars=c("MovieName",
                           "Genre"))

print("Long Format")
print(long_data)

wide_data <- dcast(long_data,
                  MovieName + Genre ~ variable,
                  value.var="value")

print("Wide Format")
print(wide_data)

write.csv(wide_data,
          "MovieReport.csv",
          row.names=FALSE)

print("Movie Report Saved")
```

```

> print(long_data)
  MovieName Genre variable value
1      Leo Action ViewerRating 4.5
2      96 Romance ViewerRating 4.8
3     Jailer Action ViewerRating 4.6
4 Love Today Comedy ViewerRating 4.4
5      Leo Action Platform Netflix
6      96 Romance Platform Prime
7     Jailer Action Platform Hotstar
8 Love Today Comedy Platform Netflix
>
> wide_data <- dcast(long_data,
+                     MovieName + Genre ~ variable,
+                     value.var="value")
>
> print("Wide Format")
[1] "Wide Format"
> print(wide_data)
  MovieName Genre ViewerRating Platform
1      96 Romance          4.8    Prime
2     Jailer Action          4.6  Hotstar
3      Leo Action          4.5  Netflix
4 Love Today Comedy          4.4  Netflix
>
> write.csv(wide_data,
+           "MovieReport.csv",
+           row.names=FALSE)
>
> print("Movie Report Saved")
[1] "Movie Report Saved"

```

11. Product Inventory Management (10 Marks)

An inventory system maintains:

- Product Details
- Stock Details
- Supplier Details

Write an R program to:

- Merge all three datasets.
- Identify products with stock less than 20 units.
- Calculate supplier-wise inventory value.
- Save the report.

```
product <- read.csv("D:/Product.csv")
```

```
stock <- read.csv("D:/Stock.csv")
```



```

supplier <- read.csv("D:/Supplier.csv")

print("Product Details")
print(product)

print("Stock Details")
print(stock)

print("Supplier Details")
print(supplier)

inventory <- merge(product,
                    stock,
                    by = "ProductID")

inventory <- merge(inventory,
                    supplier,
                    by = "SupplierID")

print("Merged Dataset")
print(inventory)

low_stock <- inventory[inventory$Stock < 20, ]

print("Products with Stock Less Than 20")
print(low_stock)

inventory$InventoryValue <-
  inventory$Stock * inventory$Price

print("Inventory Value")
print(inventory)

supplier_value <- aggregate(InventoryValue ~ SupplierName,
                             data = inventory,
                             sum)

print("Supplier-wise Inventory Value")
print(supplier_value)

write.csv(inventory,
          "InventoryReport.csv",
          row.names = FALSE)

```

```

print("Inventory Report Saved")
>
> print("Products with Stock Less Than 20")
[1] "Products with Stock Less Than 20"
> print(low_stock)
  SupplierID ProductID ProductName Stock Price SupplierName
1          S1         P1      Laptop   15 50000    ABC Traders
2          S1         P3      Tablet   18 20000    ABC Traders
>
> inventory$InventoryValue <-
+   inventory$Stock * inventory$Price
>
> print("Inventory Value")
[1] "Inventory Value"
> print(inventory)
  SupplierID ProductID ProductName Stock Price SupplierName InventoryValue
1          S1         P1      Laptop   15 50000    ABC Traders          750000
2          S1         P3      Tablet   18 20000    ABC Traders          360000
3          S2         P2      Mobile   30 25000    XYZ Suppliers          750000

```

12. COVID-19 Data Analysis (10 Marks)

A dataset contains State, Date, Confirmed Cases, Recovered Cases, Deaths.

Write an R program to:

- Read the dataset.
- Reshape it using melt().
- Calculate state-wise recovery percentage.
- Convert the data back using dcast().
- Export the final data.

```
library(reshape2)
```

```
covid <- read.csv("D:/Covid.csv")
```

```
print("COVID Dataset")
```

```
print(covid)
```

```
long_data <- melt(covid,
  id.vars = c("State", "Date"))
```

```
print("Long Format")
```

```
print(long_data)
```

```
covid$RecoveryPercentage <-
  (covid$RecoveredCases /
    covid$ConfirmedCases) * 100
```

```
print("Recovery Percentage")
```

```
print(covid)
```

```
state_recovery <- aggregate(RecoveryPercentage ~ State,
```

```

        data = covid,
        mean)

print("State-wise Recovery Percentage")
print(state_recovery)

wide_data <- dcast(long_data,
                   State + Date ~ variable,
                   value.var = "value")

print("Wide Format")
print(wide_data)

write.csv(wide_data,
          "CovidReport.csv",
          row.names = FALSE)

print("COVID Report Saved")

> print("State-wise Recovery Percentage")
[1] "State-wise Recovery Percentage"
> print(state_recovery)
      State RecoveryPercentage
1  Karnataka          91.66667
2   Kerala           94.44444
3 Tamil Nadu           95.00000
>
> wide_data <- dcast(long_data,
+                   State + Date ~ variable,
+                   value.var = "value")
>
> print("Wide Format")
[1] "Wide Format"
> print(wide_data)
      State      Date ConfirmedCases RecoveredCases Deaths
1  Karnataka 01-01-2026           1200             1100     22
2   Kerala   01-01-2026            900              850     18
3 Tamil Nadu 01-01-2026           1000              950     20
>
> write.csv(wide_data,
+          "CovidReport.csv",
+          row.names = FALSE)
>
> print("COVID Report Saved")
[1] "COVID Report Saved"

```

13. Library Management System (10 Marks)

Library maintains:

- Book Details
- Borrower Details
- Issue Details

Write an R program to:

- Merge all datasets.
- Find books issued more than five times.
- Identify overdue books.
- Save the report into a CSV file.

```
book <- read.csv("D:/Book.csv")
```

```
borrower <- read.csv("D:/Borrower.csv")
```

```
issue <- read.csv("D:/Issue.csv")
```

```
print("Book Details")
```

```
print(book)
```

```
print("Borrower Details")
```

```
print(borrower)
```

```
print("Issue Details")
```

```
print(issue)
```

```
library_data <- merge(book,  
                      issue,  
                      by = "BookID")
```

```
library_data <- merge(library_data,  
                      borrower,  
                      by = "BorrowerID")
```

```
print("Merged Dataset")
```

```
print(library_data)
```

```
issue_count <- aggregate(BookID ~ BookID,  
                          data = library_data,  
                          length)
```

```
more_issue <- issue_count[  
  issue_count$BookID > 5, ]
```

```
print("Books Issued More Than Five Times")
```

```

print(more_issue)

overdue <- library_data[
  library_data$returnDate >
  library_data$DueDate, ]

print("Overdue Books")
print(overdue)

write.csv(library_data,
          "LibraryReport.csv",
          row.names = FALSE)

print("Library Report Saved")

```

```

>
> print("Books Issued More Than Five Times")
[1] "Books Issued More Than Five Times"
> print(more_issue)
[1] 7
>
> overdue <- library_data[
+   library_data$returnDate >
+   library_data$DueDate, ]
>
> print("Overdue Books")
[1] "Overdue Books"
> print(overdue)
  BorrowerID BookID BookName IssueDate DueDate ReturnDate BorrowerName
1         BR1     B1     Java 01-01-2026 10-01-2026 12-01-2026       Arun
2         BR1     B1     Java 15-02-2026 25-02-2026 28-02-2026       Arun
6         BR3     B1     Java 01-02-2026 10-02-2026 11-02-2026       Ravi
7         BR3     B1     Java 15-03-2026 25-03-2026 27-03-2026       Ravi
>
> write.csv(library_data,
+           "LibraryReport.csv",
+           row.names = FALSE)
>
> print("Library Report Saved")
[1] "Library Report Saved"

```

14. E-Commerce Delivery Analysis (10 Marks)

Datasets contain:

- Customer Information
- Order Information
- Delivery Information

Write an R program to:

- Merge all datasets.
- Calculate average delivery time city-wise.
- Find delayed deliveries (>5 days).
- Export the final report.

```
customer <- read.csv("D:/Customer.csv")
```

```
order <- read.csv("D:/Order.csv")

delivery <- read.csv("D:/Delivery.csv")

print("Customer Details")
print(customer)

print("Order Details")
print(order)

print("Delivery Details")
print(delivery)

delivery_data <- merge(customer,
                        order,
                        by = "CustomerID")

delivery_data <- merge(delivery_data,
                      delivery,
                      by = "OrderID")

print("Merged Dataset")
print(delivery_data)

city_avg <- aggregate(DeliveryTime ~ City,
                      data = delivery_data,
                      mean)

print("City-wise Average Delivery Time")
print(city_avg)

delay <- delivery_data[
  delivery_data$DeliveryTime > 5, ]

print("Delayed Deliveries")
print(delay)

write.csv(delivery_data,
          "DeliveryReport.csv",
          row.names = FALSE)

print("Delivery Report Saved")
```

```

>
> print("City-wise Average Delivery Time")
[1] "City-wise Average Delivery Time"
> print(city_avg)
      City DeliveryTime
1 Bangalore           7
2  Chennai           3
3 Hyderabad           6
>
> delay <- delivery_data[
+   delivery_data$DeliveryTime > 5, ]
>
> print("Delayed Deliveries")
[1] "Delayed Deliveries"
> print(delay)
  OrderID CustomerID CustomerName      City OrderDate DeliveryTime
2      02         C102        Priya Bangalore 02-01-2026           7
3      03         C103         Ravi Hyderabad 03-01-2026           6
>
> write.csv(delivery_data,
+           "DeliveryReport.csv",
+           row.names = FALSE)
>
> print("Delivery Report Saved")
[1] "Delivery Report Saved"
>

```

15. College Placement Analysis (10 Marks)

Placement data contains:

- Student Details
- Company Details
- Salary Details

Write an R program to:

- Read all datasets.
- Merge them using appropriate keys.
- Calculate department-wise placement percentage.
- Identify the company offering the highest average package.
- Save the final report as a CSV file.

```

student <- read.csv("D:/Student.csv")
company <- read.csv("D:/Company.csv")
salary <- read.csv("D:/Salary.csv")
print("Student Details")
print(student)
print("Company Details")
print(company)
print("Salary Details")
print(salary)
placement <- merge(student,salary,by="StudentID")
placement <- merge(placement,company,by="CompanyID")

```

```

print("Merged Dataset")
print(placement)
dept_place <- aggregate(Placed ~ Department,data=placement,function(x) mean(x)*100)
print("Department-wise Placement Percentage")
print(dept_place)
avg_package <- aggregate(Package ~ CompanyName,data=placement,mean)
highest <- avg_package[which.max(avg_package$Package),]
print("Company with Highest Average Package")
print(highest)
write.csv(placement,"D:/PlacementReport.csv",row.names=FALSE)
print("Placement Report Saved")

```

```

> print(student)
  StudentID  Name Department Subject Marks1 Marks2 Marks3
1         101  Arun         CSE   Maths     80      75     90
2         102 Priya          IT   Maths     95      90     85
3         103  Ravi         ECE   Maths     70      65     75
4         104 Meena         CSE   Maths     88      92     84
5         105 Kiran          IT   Maths     78      80     82

```

```

> print("Company Details")
[1] "Company Details"
> print(company)
  CompanyID CompanyName
1         C1          TCS
2         C2      Infosys
3         C3    Accenture

```

```

> print("Salary Details")
[1] "Salary Details"
> print(salary)
  EmpID BasicPay Allowance
1  E101    40000    12000
2  E102    55000    15000
3  E103    45000    10000
4  E104    60000    18000
5  E105    38000     9000

```

```

> placement <- merge(student,salary,by="StudentID")

```

RUBRICS FOR CALCULATION

Analytical Problem Solving

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation